
Probato AI

Pre-Launch Testing Protocol

Comprehensive Testing Checklist for All 34 Milestones

Phase 1 through Phase 6

Version 1.0 | May 2026

Document Classification: Internal | Pre-Production

Document Purpose	Scope
This document provides a systematic testing protocol for validating every milestone of the Probato AI platform prior to commercial launch. Each milestone includes specific test procedures, measurable success criteria, and sign-off fields. Testers should complete every test item, record results, and flag blockers before proceeding to the next phase.	6 phases, 34 milestones, 200+ individual test items covering: authentication, browser automation, LLM integration, billing, security, accessibility, visual regression, team collaboration, enterprise SSO, plugin architecture, synthetic monitoring, and more.

Table of Contents

How to Use This Document	4
Phase 1: Core Platform	5
M1: GitHub OAuth and Dashboard	5
M2: Docker Sandbox API	5
M3: Browser Automation	6
M4: LLM Code Analysis	6
M5: Test Executor Agent	7
M6: Feature Discovery Agent	7
M7: Smart Test Generation and Auto-Heal	8
M8: Reporting and Project Management	8
Phase 2: Professional Features	9
M9: GitHub CI/CD Integration	9
M10: Scheduled and Recurring Tests	10
M11: Visual Regression Testing	10
M12: Notifications and Alerts	11
M13: Billing and Subscription	12
M14: Public API and Developer SDK	12
Phase 3: User Experience	14
M15: Live Test Execution View	14
M16: Fix Suggestion and Approval Workflow	14
M17: Security and Accessibility Testing	14

M18: Enhanced Landing Page and Onboarding	15
M19: Team Collaboration and Test Sharing	15
M20: Test Intelligence Dashboard	16
Phase 4: Advanced Testing Agents	17
M21: Media Verification (Image and Video)	17
M22: Audio Verification and Whisper Integration	17
M23: Active Security v1 (XSS and Auth Probing)	17
M24: Active Security v2 (API, CSRF, Rate Limiting, IDOR)	18
M25: Multi-Sandbox Orchestrator	18
M26: Cross-Device Messaging Testing	19
M27: Call Flow Testing	19
M28: Payment Flow Testing	19
Phase 5: Enterprise Intelligence	20
M29: AI Test Intelligence Engine	20
M30: Self-Healing Tests v2 and Auto-Maintenance	20
M31: Synthetic Monitoring and Performance	21
M32: Enterprise SSO, Audit, and Compliance	21
M33: Plugin Architecture and Marketplace	22
M34: Integration and Polish	22
Pre-Launch Hardening Checklist	23
Final Launch Sign-Off	24

How to Use This Document

This testing protocol is designed to be used as a hands-on checklist while you wire and configure Probato AI for production. Each phase is presented in the order it was built, reflecting the dependency chain: you must validate Phase 1 before Phase 2 features can reliably be tested, and so on. Each milestone contains a table of test items with three columns: the test name, the exact procedure to follow, and the measurable success criteria that must be met for the milestone to be considered complete.

Before beginning, ensure you have the following prerequisites in place: a deployed instance of Probato AI accessible at your Vercel URL, a GitHub account for OAuth testing, a Neon PostgreSQL database with the latest schema applied, access to your LLM provider API (OpenAI recommended as primary), and a Stripe account in test mode for billing validation. For the most accurate results, test against a production-like environment rather than a local development server.

Prerequisite	How to Verify	Required For
Deployed Vercel instance	Visit https://probato-ai.vercel.app and confirm landing page loads	All phases
GitHub OAuth configured	Click Sign In on landing page; GitHub login dialog appears	Phase 1+
Database schema applied	Run <code>prisma db push</code> or <code>prisma migrate deploy</code> ; no errors	Phase 1+
LLM API key active	Set <code>LLM_API_KEY</code> and <code>LLM_API_URL</code> in Vercel env vars	Phase 1 M4+
Stripe test mode	Set <code>PAYMENT_GATEWAY=stripe</code> and <code>STRIPE_SECRET_KEY</code> in Vercel	Phase 2 M13+
Browserless/Chromium	Set <code>BROWSERLESS_TOKEN</code> or verify Sparticuz fallback works	Phase 1 M3+
Resend email API	Set <code>RESEND_API_KEY</code> and <code>NOTIFICATION_EMAIL_FROM</code> in Vercel	Phase 2 M12+

Result Codes

Code	Meaning	When to Use
PASS	Test passed, success criteria fully met	All criteria satisfied, no anomalies
FAIL	Test failed, success criteria not met	Any criteria unmet, functional break
PARTIAL	Partially working, some criteria unmet	Core function works but edge cases fail
BLOCKED	Cannot test due to dependency failure	Prerequisite not available (e.g., no LLM key)
SKIP	Intentionally skipped with documented reason	Feature not relevant to current test environment

Important: A milestone is only considered COMPLETE when ALL test items pass (PASS). If any item is FAIL or BLOCKED, the milestone is incomplete and must be resolved before proceeding to the next phase.

Record all PARTIAL and FAIL results with specific error details in the notes column or on a separate defect log.

Phase 1: Core Platform

Phase 1 establishes the foundational infrastructure: authentication, sandbox environments, browser automation, LLM-powered code analysis, test execution, feature discovery, smart test generation with auto-healing, and reporting. These are the most critical milestones because every subsequent phase depends on them. If Phase 1 is not rock-solid, nothing built on top of it will be reliable.

M1: GitHub OAuth and Dashboard

This milestone validates that users can authenticate via GitHub OAuth, that their profile data is correctly stored in the database, and that the dashboard renders with accurate information. It is the gateway to every other feature in the platform.

Test Item	Test Procedure	Success Criteria
GitHub OAuth Sign-In	1. Navigate to the landing page. 2. Click "Sign In" or "Get Started". 3. Complete GitHub OAuth flow. 4. Verify redirect to /dashboard.	User is authenticated and redirected to /dashboard; session cookie is set; user record exists in database with GitHub profile data (name, email, avatar).
Dashboard Rendering	1. After sign-in, observe the dashboard page. 2. Verify user name, avatar, and email display correctly. 3. Check project list loads (empty state is acceptable).	Dashboard renders without JavaScript errors; user info displays correctly; no console errors in browser DevTools; page load time under 3 seconds.
Session Persistence	1. Sign in and navigate to dashboard. 2. Close the browser tab. 3. Reopen the same URL. 4. Verify session is still active.	User remains authenticated after browser restart; no need to re-login within the 30-day session window.
Sign-Out	1. Click sign-out button on dashboard. 2. Verify redirect to landing page. 3. Attempt to navigate to /dashboard.	User is redirected to landing page; accessing /dashboard redirects to sign-in; session cookie is cleared.
NEXTAUTH_SECRET Set	1. Check Vercel environment variables. 2. Verify NEXTAUTH_SECRET is set to a cryptographically random string (not the fallback value).	NEXTAUTH_SECRET is a 32+ character random string; no fallback secret is in use; session tokens cannot be forged.

M2: Docker Sandbox API

The Docker Sandbox allows users to spin up isolated container environments for testing their projects. On Vercel (serverless), Docker is unavailable, so the endpoints should return a graceful 503 error. If you have a remote Docker host, test full functionality.

Test Item	Test Procedure	Success Criteria
Sandbox Creation (Docker Available)	1. POST /api/sandbox with a GitHub repo URL. 2. Verify container starts. 3. Check container status via GET /api/sandbox/[id].	Container starts within 60 seconds; status shows "running"; framework is auto-detected correctly (Next.js, Vite, etc.).
Sandbox Graceful Failure (No Docker)	1. POST /api/sandbox from Vercel deployment. 2. Verify 503 response. 3. Check that error message is helpful (suggests Docker or remote host).	Returns HTTP 503 with a clear message; no server crash or unhandled error; user understands why sandbox is unavailable.
Sandbox Destroy	1. DELETE /api/sandbox/[id] with a running container ID. 2. Verify container stops and is removed.	Container is stopped and removed; GET /api/sandbox/[id] returns 404; Docker network is cleaned up.

M3: Browser Automation

Browser automation is the core capability that drives all test execution. This milestone validates that the 4-tier browser strategy (Browserless, WebSocket endpoint, Sparticuz for Vercel, local Chromium) works correctly and can navigate, interact, and capture screenshots from web pages.

Test Item	Test Procedure	Success Criteria
Browser Launch (Cloud)	1. Set BROWSERLESS_TOKEN in env vars. 2. Trigger any test run. 3. Verify browser connects to Browserless cloud.	Browser connects successfully; test executes; no timeout on browser launch; screenshot is captured.
Browser Launch (Sparticuz Fallback)	1. Remove BROWSERLESS_TOKEN. 2. Trigger a test run on Vercel. 3. Verify Sparticuz Chromium launches.	Sparticuz Chromium initializes; test executes; performance may be slower but functionally correct.
Browser Check Endpoint	1. GET /api/browser/check. 2. Verify it returns browser availability status.	Returns 200 with { available: true, strategy: "..."}; strategy matches the expected tier based on env vars.

M4: LLM Code Analysis

The LLM integration powers feature discovery, test generation, fix suggestions, and intelligence features. This milestone validates that the LLM API connection works, that structured JSON output is correctly parsed, and that fallback strategies function when the primary LLM is unavailable.

Test Item	Test Procedure	Success Criteria
LLM API Connection	1. Set LLM_API_URL and LLM_API_KEY in Vercel env. 2. Trigger feature discovery on a public website. 3. Verify LLM response is received and parsed.	LLM returns valid JSON; response contains expected fields; no timeout or 401 error; response time under 30 seconds for standard analysis.
Structured Output Parsing	1. Run feature discovery on a known website. 2. Verify discovered features are structured as Feature objects with name, type, priority, selector.	Each feature has all required fields; selectors are valid CSS or XPath; priority values are 1-5; no null or undefined fields in parsed output.

Test Item	Test Procedure	Success Criteria
LLM Fallback	1. Set LLM_API_KEY to an invalid value. 2. Trigger feature discovery. 3. Verify rule-based fallback produces results.	System does not crash; fallback produces basic features from DOM analysis; response indicates fallback was used; user is informed of degraded quality.

M5: Test Executor Agent

The Test Executor Agent is the engine that runs browser interaction sequences. It supports 18 action types (navigate, click, fill, select, check, uncheck, submit, press, wait, waitForSelector, waitForNavigation, screenshot, scroll, hover, assertText, assertVisible, assertUrl, readText) and 6 selector strategies. This milestone validates that tests execute end-to-end with accurate results.

Test Item	Test Procedure	Success Criteria
Smoke Test Preset	1. Enter a public URL (e.g., https://example.com). 2. Select "Smoke" preset. 3. Click "Run Test". 4. Wait for completion.	Test completes with all steps passing; screenshots captured for each step; duration displayed; TestRun record created in database.
Navigation Preset	1. Enter a multi-page website URL. 2. Select "Navigation" preset. 3. Run test.	Navigation actions execute; page transitions detected; no false failures; URL assertions pass for each page visited.
Form Fill Preset	1. Enter a URL with a form (e.g., a contact page). 2. Select "Form" preset. 3. Run test.	Form fields are filled; submit action executed; no selector failures; form submission response captured.
Custom Action Sequence	1. Enter a URL. 2. Define custom actions: navigate, click, fill, assert. 3. Run test.	Custom actions execute in order; each step has pass/fail status; failure screenshots captured; subsequent steps are skipped after failure.
Test Run Persistence	1. Run a test. 2. Navigate to Test Runs history. 3. Verify the run appears with correct step details.	TestRun record exists with all TestResult entries; step order preserved; screenshots accessible; status matches execution result.

M6: Feature Discovery Agent

Feature Discovery uses LLM analysis and DOM inspection to automatically identify testable features on a web page. This is one of the most valuable features because it eliminates the need for manual test planning. The agent should detect navigation links, forms, interactive elements, and content sections, then classify them by type and priority.

Test Item	Test Procedure	Success Criteria
Feature Discovery on Public Site	1. Enter a public website URL. 2. Click "Discover Features". 3. Wait for analysis.	Features are discovered and listed; each feature has name, type (navigation/form/interactive/content), priority (1-5), and selector; minimum 3 features found for a standard website.
Feature Persistence	1. Discover features on a project. 2. Navigate away. 3. Return to project. 4. Verify features are still listed.	Feature records persist in database; selector fields are intact; no duplicate features created on re-discovery of same page.
Credit Deduction	1. Check credit balance before discovery. 2. Run feature discovery. 3. Check balance after.	6 credits deducted; CreditTransaction record created; balance reflects deduction; 402 returned if insufficient credits.

M7: Smart Test Generation and Auto-Heal

Smart Test Generation converts discovered features into executable Playwright test code. Auto-Heal detects selector-based test failures and automatically finds alternative selectors to repair broken tests. The Dependency Graph provides topological ordering and impact analysis for test execution.

Test Item	Test Procedure	Success Criteria
Playwright Test Generation	1. Discover features on a project. 2. Click "Generate Tests". 3. View generated code.	Valid Playwright test code generated; test.describe blocks present; locators use correct Playwright syntax; code is syntactically valid TypeScript.
Auto-Heal on Broken Selector	1. Run a test that fails due to a selector change. 2. Click "Auto-Heal". 3. Verify healed selector is found.	Auto-heal identifies alternative selectors; confidence score is above 0.5; healed test passes on re-execution; TestCase updated with autoHealed=true.
Dependency Graph	1. Click "Test Order" on a project with multiple features. 2. Verify topological sort is displayed.	Features ordered by dependencies; parallel groups identified; cycle detection works if circular dependencies exist; impact analysis shows affected tests.
Credit Enforcement	1. Check credits. 2. Generate tests (5 credits). 3. Auto-heal (8 credits, Pro+ only). 4. Verify Free plan gets 403 for auto-heal.	Credits deducted correctly; Free plan blocked from auto-heal; Pro+ plan can access auto-heal; proper 402/403 responses returned.

M8: Reporting and Project Management

Reporting provides project-level analytics including feature breakdowns, test run trends, pass/fail statistics, and data export. The Project Detail Page consolidates all project data into a single view.

Test Item	Test Procedure	Success Criteria
Project Detail Page	1. Click "Details" on a project card. 2. Verify all sections render.	Stats cards display correct counts; feature list with expand/collapse; test run history with step details; "Discover" and "Generate" buttons work.
Reports API - JSON	1. GET /api/reports?projectId=X&format=json. 2. Verify response structure.	Returns project summary with features by type/priority; test run trends; per-feature results aggregation; all fields populated.
Reports API - CSV	1. GET /api/reports?projectId=X&format=csv. 2. Download and open in spreadsheet.	Valid CSV file; headers match expected columns; data rows correspond to test results; file downloads successfully.

Phase 1 Completion Criteria

All 8 milestones (M1 through M8) must have every test item marked PASS. The platform must support the complete flow: sign in, create project, discover features, generate tests, execute tests, view results, auto-heal broken tests, and export reports. No BLOCKED or FAIL items are acceptable for Phase 1 completion.

Phase Sign-Off	Value
Phase Name	Phase 1: Core Platform
Tester Name	_____
Date Tested	____ / ____ / ____
All Milestones Passed	Yes / No
Blockers Found	_____
Signature	_____

Phase 2: Professional Features

Phase 2 adds the features that transform Probato from a personal tool into a professional platform: GitHub CI/CD integration, scheduled testing, visual regression detection, multi-channel notifications, billing with real payment processing, and a public API with SDK. These features are what users pay for.

M9: GitHub CI/CD Integration

GitHub CI/CD integration allows Probato to automatically run tests when code is pushed or pull requests are opened. This requires a GitHub App with webhook configuration. Note: this milestone can only be fully tested after creating and registering a GitHub App.

Test Item	Test Procedure	Success Criteria
GitHub App Webhook	1. Configure GitHub App with webhook URL pointing to /api/webhooks/github. 2. Push a commit to a watched repository. 3. Verify webhook is received.	Webhook received and processed; push event triggers test run; webhook signature verification passes; WebhookEvent record created in database.
PR Comment Posting	1. Open a pull request on a watched repository. 2. Wait for test execution. 3. Check PR for test result comment.	Test result comment appears on PR; comment includes pass/fail summary; check run status updated to success/failure; comment is updated on subsequent pushes.
Installation Management	1. Install GitHub App on a repository. 2. Check /api/installations. 3. Toggle repository CI/CD enablement.	Installations listed with status; repositories shown per installation; enable/disable toggle works; disabled repos do not trigger test runs.

M10: Scheduled and Recurring Tests

Scheduled tests allow users to define cron-based recurring test runs. The cron trigger endpoint needs to be called by Vercel Cron or an external scheduler to execute due schedules.

Test Item	Test Procedure	Success Criteria
Schedule Creation	1. Create a schedule with name, URL, preset, and cron expression (e.g., "0 9 * * 1-5"). 2. Verify schedule is listed.	Schedule created with correct nextRunAt; cron description is human-readable; schedule appears in dashboard list with Active badge.
Schedule Pause/Resume	1. Click pause on an active schedule. 2. Verify status changes to Paused. 3. Click enable. 4. Verify Active status.	Schedule toggles between Active/Paused; nextRunAt recalculated on resume; no test runs triggered while paused.
Cron Trigger Endpoint	1. POST /api/cron/run-schedules with CRON_SECRET header. 2. Verify due schedules are executed.	Due schedules executed; test runs created; schedule lastRunAt updated; endpoint returns summary of executed schedules.

M11: Visual Regression Testing

Visual regression testing captures baseline screenshots and compares subsequent screenshots to detect visual changes. This is a Pro+ feature that uses pixel-level comparison with configurable thresholds.

Test Item	Test Procedure	Success Criteria
Baseline Capture	1. Enter a URL and name. 2. Click "Capture Baseline". 3. Wait for screenshot.	Baseline screenshot captured; VisualBaseline record created; screenshot is a valid PNG; baseline name and URL stored correctly.

Test Item	Test Procedure	Success Criteria
Visual Comparison - No Change	1. Capture a baseline. 2. Compare the same URL. 3. Check diff result.	Mismatch percentage is 0% (or near 0% for anti-aliasing); diff image is transparent; status is "match" or equivalent.
Visual Comparison - With Change	1. Capture a baseline. 2. Modify the page (or use a different URL). 3. Compare. 4. Check diff.	Mismatch percentage is above threshold; diff image highlights changed pixels in red; VisualDiff record created with status "pending_review".
Approve/Reject Diff	1. View a pending diff. 2. Click "Accept". 3. Verify baseline updates. 4. Create another diff and click "Reject".	Accept: baseline screenshot updated to current; Reject: original baseline preserved; diff status changes to approved/rejected; notification dispatched.
Pro Plan Gate	1. Test with Free plan account. 2. Attempt visual comparison.	Returns 403 with requiredPlan: "pro"; Free users cannot compare; 3 credits deducted for Pro users; feature works on Pro+ plans.

M12: Notifications and Alerts

The notification system supports in-app, email (Resend), Slack, Discord, and custom webhook channels. Notifications are triggered by test results, visual diffs, schedule completions, and auto-heal events.

Test Item	Test Procedure	Success Criteria
In-App Notifications	1. Run a test that fails. 2. Check notification bell icon. 3. Verify notification appears.	Bell icon shows unread count; notification panel displays message; clicking marks as read; "Mark all read" works; dismiss button functional.
Slack Channel Integration	1. Add a Slack webhook URL in notification channels. 2. Trigger a test failure. 3. Verify Slack message delivery.	Slack message received with test result details; Block Kit formatting renders; auto-verification message sent on channel creation; channel listed as verified.
Email Notification (Resend)	1. Configure RESEND_API_KEY and NOTIFICATION_EMAIL_FROM. 2. Add email channel. 3. Trigger a test failure. 4. Check inbox.	Email received with test result; email contains action URL; sender address matches NOTIFICATION_EMAIL_FROM; email renders correctly in Gmail/Outlook.
Notification Preferences	1. Open notification preferences. 2. Toggle event types. 3. Verify changes persist.	Preferences save correctly; toggling off email stops email delivery; in-app notifications always available for enabled event types.

M13: Billing and Subscription

Billing is the revenue engine of Probato. It supports a hybrid model (subscription + pay-per-use credits) with three payment gateways: Mock (dev), Stripe (global), and Pystack (Africa/M-Pesa). This milestone **MUST** be tested with Stripe test mode before launch.

Test Item	Test Procedure	Success Criteria
Free Plan Default	1. Sign up a new user. 2. Check billing page. 3. Verify Free plan is active.	Free plan active with 20 credits; credit balance shows 20; no payment method required; project limit is 1.
Pro Plan Upgrade (Stripe Test)	1. Click "Upgrade to Pro". 2. Complete Stripe Checkout with test card 4242424242424242. 3. Verify plan activation.	Stripe Checkout page opens; payment succeeds; plan changes to Pro; 200 credits added; Subscription record created with stripeCustomerId.
Credit Pack Purchase	1. Click "Buy" on a credit pack. 2. Complete checkout. 3. Check credit balance.	Credits added immediately; CreditTransaction shows "credit" type; balance reflects addition; pack discount applied correctly.
Credit Deduction on Actions	1. Run a feature discovery (6 credits). 2. Generate tests (5 credits). 3. Check credit history.	Each action deducts correct credit amount; CreditTransaction records created; balance updates in real-time; low balance warning appears below 20%.
Insufficient Credits (402)	1. Deplete all credits. 2. Attempt feature discovery.	Returns HTTP 402 with creditsRequired and creditsBalance; action is not executed; user is prompted to buy credits or upgrade plan.
Plan Cancellation	1. Cancel Pro subscription. 2. Verify downgrade to Free at period end.	Subscription cancelAtPeriodEnd set to true; access continues until period end; then reverts to Free plan with 20 credits; Stripe portal accessible.
Webhook Processing	1. Use Stripe CLI to send test webhook events. 2. Verify /api/billing/webhook handles them.	Webhook signature verified; subscription status updated on checkout.session.completed; invoice.paid event processed; 200 response for valid webhooks; 400 for invalid signatures.

M14: Public API and Developer SDK

The public API (versioned at /api/v1/) provides programmatic access to all Probato features. The TypeScript SDK wraps the API for developer convenience. API keys use script hashing and scope-based access control.

Test Item	Test Procedure	Success Criteria
API Key Creation	1. Open API Keys dialog. 2. Create a key with "read" scope. 3. Copy the key.	Key is displayed once (never again); key is script-hashed in database; ApiKey record created with correct scope; usage tracking starts.
API Key Authentication	1. Make a GET /api/v1/projects request with Bearer token. 2. Verify response.	Request succeeds with valid key; 401 returned for invalid key; scope-checked: read key cannot POST; write key can POST.
Rate Limiting	1. Make 15 rapid requests with Free plan API key. 2. Check response headers.	Rate limit headers present (X-RateLimit-Limit, X-RateLimit-Remaining); 429 returned after 10 requests (Free limit); Pro allows 60/min; Team 120/min.
SDK Functionality	1. Install @probatosdk (or use local path). 2. Initialize with API key. 3. Call projects.list(), discovery.run(), etc.	SDK authenticates; methods return typed responses; error classes work (AuthenticationError, RateLimitError, InsufficientCreditsError); retry on 429.
OpenAPI Spec	1. GET /api/v1/docs. 2. Verify OpenAPI 3.0.3 specification.	Valid OpenAPI JSON; all endpoints documented; request/response schemas defined; authentication methods described; examples provided.

Phase 2 Completion Criteria

All 6 milestones (M9 through M14) must pass. The billing flow must work end-to-end with Stripe test mode. The API must authenticate correctly and enforce rate limits. At minimum, at least one notification channel (Slack or email) must deliver successfully.

Phase Sign-Off	Value
Phase Name	Phase 2: Professional Features
Tester Name	_____
Date Tested	____ / ____ / ____
All Milestones Passed	Yes / No
Blockers Found	_____
Signature	_____

Phase 3: User Experience

Phase 3 focuses on the user-facing experience: real-time test execution, fix suggestions, security and accessibility testing, an enhanced onboarding flow, team collaboration, and a test intelligence dashboard. These features differentiate Probato from basic testing tools.

M15: Live Test Execution View

Test Item	Test Procedure	Success Criteria
Live Test Stream	1. Enter a URL in Live Test View. 2. Click "Run". 3. Observe real-time step updates.	Steps appear one by one in real-time; status icons update (running, passed, failed); screenshot viewer shows current page; progress bar advances; no WebSocket errors.
Step Screenshot Navigation	1. After a live test completes, click Previous/Next to browse step screenshots.	Each step shows its screenshot; navigation is responsive; step details expand; console errors and network requests visible in diagnostics panel.
Live Test Cancellation	1. Start a live test. 2. Click "Cancel" mid-execution.	Test stops immediately; partial results are saved; TestRun status set to "cancelled"; no zombie browser processes.

M16: Fix Suggestion and Approval Workflow

Test Item	Test Procedure	Success Criteria
Fix Suggestion Generation	1. Run a test that fails. 2. View fix suggestions for the failed step.	FixSuggestion record created; confidence score provided; original and suggested code shown; reasoning explained; diff display available.
Fix Approval and Application	1. Review a pending fix suggestion. 2. Click "Approve". 3. Verify fix is applied.	Fix status changes to "approved"; suggested code replaces original; status updates to "applied"; test can be re-run with fixed code.
Fix Rejection	1. Click "Reject" on a fix suggestion. 2. Add a review note.	Fix status changes to "rejected"; review note saved; original code unchanged; no automatic re-application.

M17: Security and Accessibility Testing

Test Item	Test Procedure	Success Criteria
Passive Security Scan	1. Enter a URL. 2. Click "Security Scan". 3. Review findings.	Scan completes; security score calculated; findings categorized by severity (critical/high/medium/low/info); recommendations provided; SecurityScan record saved.
Accessibility Audit	1. Enter a URL. 2. Click "A11y Audit". 3. Review WCAG findings.	Audit runs against WCAG 2.1 AA; violations listed with impact level; element selectors provided for each issue; remediation suggestions given; A11yAudit record persisted.
Credit Deduction	1. Run security scan (4 credits). 2. Run a11y audit (5 credits). 3. Verify credit deductions.	Correct credits deducted for each action; transactions recorded; 402 returned when credits insufficient.

M18: Enhanced Landing Page and Onboarding

Test Item	Test Procedure	Success Criteria
Onboarding Wizard Flow	1. Sign in as a new user. 2. Complete all 5 onboarding steps. 3. Reach dashboard.	Welcome step shows feature highlights; repo connection step creates project; discovery step auto-runs feature discovery; first test step executes a test; completion step shows summary; redirected to dashboard after completion.
Onboarding Skip	1. Click "Skip for now" on the welcome step. 2. Verify redirect to dashboard.	User skips to dashboard; onboarding state saved as "skipped"; checklist still visible on dashboard; can resume onboarding later.
Landing Page Content	1. Visit the landing page as an unauthenticated user. 2. Verify all sections.	Hero section with CTA; feature cards (9 features); pricing section with 3 tiers; FAQ accordion; footer with links; all CTAs redirect to /onboarding.

M19: Team Collaboration and Test Sharing

Test Item	Test Procedure	Success Criteria
Team Creation (Team Plan)	1. Upgrade to Team plan. 2. Create a team with name and description.	Team created; creator is owner; TeamMember record exists; team appears in collaboration panel; team plan gate enforced (Free/Pro get blocked).

Test Item	Test Procedure	Success Criteria
Team Invitation	1. Click "Invite Member". 2. Enter an email. 3. Select a role. 4. Send invite.	TeamInvitation record created with unique token; invitation email sent (if Resend configured); token expires in 7 days; pending invitations listed.
Project Sharing	1. Click share on a project. 2. Enter email and select permission (view/edit/admin). 3. Verify shared user can access.	SharedProject record created; shared user sees project in "Shared with You"; permissions enforced: view=read-only, edit=can run tests, admin=full access.
Comments	1. Add a comment on a test run. 2. Reply to the comment. 3. Resolve it.	Comment created with author and timestamp; reply nested correctly; resolve toggle works; resolved comments visually distinguished.

M20: Test Intelligence Dashboard

Test Item	Test Procedure	Success Criteria
Health Score Display	1. Open Test Intelligence panel. 2. Verify health score gauge.	Health score calculated (0-100); gauge renders with correct color; score reflects actual pass rate minus flake and duration penalties.
Flaky Test Detection	1. Run tests multiple times with intermittent failures. 2. Check flaky tests section.	Flaky tests identified; pass/fail pattern shown; flake rate calculated; tests classified as stable/flaky/failing/unknown.
Recommendations Engine	1. View AI-generated recommendations. 2. Verify actionable insights.	Recommendations are priority-ranked; each has a specific action; critical recommendations appear first; recommendations reference real data.

Phase 3 Completion Criteria

All 6 milestones (M15 through M20) must pass. The onboarding flow must work for new users end-to-end. Team features must work on the Team plan. The intelligence dashboard must display real data from test runs, not placeholder values.

Phase Sign-Off	Value
Phase Name	Phase 3: User Experience
Tester Name	_____
Date Tested	____ / ____ / ____

All Milestones Passed	Yes / No
Blockers Found	_____
Signature	_____

Phase 4: Advanced Testing Agents

Phase 4 introduces specialized testing agents: media verification (images, video, audio), active security probing (XSS, authentication, API, CSRF, rate limiting, IDOR), and multi-device orchestration for cross-platform testing scenarios.

M21: Media Verification (Image and Video)

Test Item	Test Procedure	Success Criteria
Image Verification	1. Enter a URL with images. 2. Click "Verify Media". 3. Review image checks.	Broken images (0x0) detected; hidden images flagged; missing alt text reported; distorted images identified; overall image score calculated (0-100).
Video Verification	1. Enter a URL with video elements. 2. Verify media with video checks enabled.	Video readyState checked; playback errors detected; source availability verified; frame capture works at key timestamps; video score calculated.

M22: Audio Verification and Whisper Integration

Test Item	Test Procedure	Success Criteria
Audio Element Verification	1. Enter a URL with audio elements. 2. Verify media with audio checks enabled.	Audio elements detected; readyState checked; muted/hidden elements flagged; audio score calculated; embedded audio (iframes) detected.
Whisper Transcription	1. Enable "Transcribe Audio" toggle. 2. Run media verification.	Audio sources transcribed via Whisper API; transcription confidence score provided; silent audio flagged as medium severity; transcription stored in verification record.

M23: Active Security v1 (XSS and Auth Probing)

Test Item	Test Procedure	Success Criteria
Reflected XSS Probe	1. Enter a URL. 2. Run security probe with "standard" depth. 3. Check XSS findings.	XSS test payloads injected in URL params and form inputs; reflected/sanitized status reported; DOM-based XSS sinks detected; XSS score calculated.
Auth Probe	1. Run security probe. 2. Check auth findings.	CSRF token detection; session cookie analysis; open redirect testing; rate limit testing on auth endpoints; auth score calculated.
Credit Deduction	1. Run security probe (6 credits). 2. Verify deduction.	6 credits deducted; SecurityProbe record created; notification dispatched.

M24: Active Security v2 (API, CSRF, Rate Limiting, IDOR)

Test Item	Test Procedure	Success Criteria
API Security Probe	1. Enter a URL with API endpoints. 2. Run API probe. 3. Check findings.	API endpoints discovered; unauthenticated endpoints flagged; verbose errors detected; CORS misconfiguration reported; API score calculated.
CSRF and Rate Limit Probe	1. Run API probe. 2. Check CSRF and rate limit findings.	Missing CSRF tokens on state-changing endpoints; weak rate limits detected; SameSite cookie attribute checked; scores calculated.
IDOR Probe	1. Run API probe with "deep" depth. 2. Check IDOR findings.	Sequential ID patterns detected; cross-ID access tested; mass assignment probed; exposed list endpoints flagged; IDOR score calculated.

M25: Multi-Sandbox Orchestrator

Test Item	Test Procedure	Success Criteria
Orchestrated Session Creation	1. Enter a URL. 2. Select "messaging" scenario. 3. Create session.	Session created with multiple sandbox agents; agents configured with default actions; SyncEvent records created; session status is "running".
Session Abort	1. Start an orchestrated session. 2. Click "Abort". 3. Verify cleanup.	Session status changes to "aborted"; running agents stop; partial results saved; no orphan browser processes.

M26: Cross-Device Messaging Testing

Test Item	Test Procedure	Success Criteria
Messaging Test Scenario	1. Enter a messaging app URL. 2. Select "messaging" scenario. 3. Run test.	Two agents execute message send/receive; notification toast detection works; latency measured; messaging score calculated; 12 credits deducted.

M27: Call Flow Testing

Test Item	Test Procedure	Success Criteria
Call Flow Test Scenario	1. Enter a calling app URL. 2. Select "call-flow" scenario. 3. Run test.	Caller dials, callee receives; ring latency measured; connection verified; mute/speaker toggle tested; call flow score calculated; 12 credits deducted.

M28: Payment Flow Testing

Test Item	Test Procedure	Success Criteria
Payment Flow - Success	1. Enter an e-commerce URL. 2. Select "payment" scenario. 3. Run with Stripe test card 4242.	Checkout pipeline completes; payment succeeds; webhook delivery verified; payment score calculated; 15 credits deducted.
Payment Flow - Decline	1. Run payment flow with decline test card 4000000000000002.	Card decline detected; error message captured; payment score reflects failure; no false success reported.
Payment Flow - 3D Secure	1. Run payment flow with 3DS test card 4000002760003184.	3DS challenge detected; flow handles authentication step; result correctly captured regardless of 3DS outcome.

Phase 4 Completion Criteria

All 8 milestones (M21 through M28) must pass. Security probes must detect at least one genuine finding on a deliberately vulnerable test site. Multi-device orchestration must successfully coordinate at least 2 browser agents. Payment flow must handle all Stripe test card scenarios.

Phase Sign-Off	Value
Phase Name	Phase 4: Advanced Testing Agents

Tester Name	_____
Date Tested	_____ / _____ / _____
All Milestones Passed	Yes / No
Blockers Found	_____
Signature	_____

Phase 5: Enterprise Intelligence

Phase 5 delivers enterprise-grade intelligence: AI-powered test selection and prioritization, enhanced self-healing with auto-maintenance, synthetic monitoring with performance baselines, enterprise SSO and audit compliance, plugin architecture with marketplace, and integration polish.

M29: AI Test Intelligence Engine

Test Item	Test Procedure	Success Criteria
Dependency Graph Build	1. On a project with multiple features, run dependency analysis. 2. Verify graph is constructed.	Dependencies resolved from feature names; topological sort produces valid order; cycle detection works; parallel groups identified; 3 credits deducted.
Smart Test Selection	1. Run smart selection for a code change. 2. Verify only affected tests are selected.	Selection reduces test count by 60-80% from full suite; all impacted tests included; unrelated tests excluded; selection reasoning provided.
Flakiness Analysis	1. Run flakiness analysis on a project with historical test data. 2. Check classification.	Tests classified as stable/flaky/failing/unknown; flake rate calculated per test; 10 credits deducted; FlakinessReport record created.
Impact Analysis	1. Run impact analysis on a specific feature change. 2. Verify impact scores.	Direct and transitive dependents identified; risk levels assigned; 20 credits deducted; ImpactAnalysisResult record created.

M30: Self-Healing Tests v2 and Auto-Maintenance

Test Item	Test Procedure	Success Criteria
Selector Repair	1. Run a test with a broken selector. 2. Trigger self-heal v2. 3. Verify repair at confidence ≥ 0.85 auto-applies.	Broken selector detected; alternative found with confidence score; auto-repair applies if confidence ≥ 0.85 ; SelectorRepair record created; 8 credits deducted.
Maintenance Scan	1. Run maintenance scan on a project with generated test code. 2. Check categories.	Deprecation patterns detected (page.waitFor, page.\$); assertion drift flagged; step staleness identified; code quality issues found; 6 credits deducted.
Deprecation Detection	1. Check deprecation reports for Playwright API patterns.	5 Playwright deprecation patterns detected; replacement suggestions provided; code snippets show before/after; deprecation severity assigned.

M31: Synthetic Monitoring and Performance

Test Item	Test Procedure	Success Criteria
Checkpoint Creation	1. Create a synthetic checkpoint with URL and assertions. 2. Verify it is listed.	SyntheticCheckpoint record created; assertion configuration saved; checkpoint appears in monitoring dashboard; next run time calculated.
Checkpoint Execution	1. Manually trigger a checkpoint run. 2. Verify results.	Checkpoint runs against target URL; response time measured; assertions evaluated; CheckpointResult record created; performance metrics captured.
Performance Baseline	1. Run multiple checkpoint executions. 2. Check performance baseline.	PerformanceBaseline established from historical data; regression detection works; alerts triggered when performance degrades beyond threshold.

M32: Enterprise SSO, Audit, and Compliance

Test Item	Test Procedure	Success Criteria
SSO Configuration (SAML)	1. Create an SSO configuration with SAML protocol. 2. Test connection.	SSO configuration saved; SP metadata generated; test connection returns status; domain restrictions enforced; auto-provisioning toggle works.

Test Item	Test Procedure	Success Criteria
Audit Log	1. Perform several actions (run test, create project, change settings). 2. Check audit log.	All actions logged; before/after snapshots captured; hash chain intact; severity levels assigned; filterable by action, resource, user, date.
Audit Chain Verification	1. Click "Verify Chain Integrity". 2. Confirm all hashes are valid.	SHA-256 hash chain verified; no tampering detected; verification result stored; tampered entries (if any) are flagged.
RBAC Permission Check	1. Create a custom permission policy. 2. Test permission check.	5 default policies auto-seeded; custom policy created; permission check returns allowed/denied with source; role-to-permission mapping works.

M33: Plugin Architecture and Marketplace

Test Item	Test Procedure	Success Criteria
Plugin Registration	1. Create a plugin with name, version, and entry point. 2. Verify it is listed.	Plugin record created; version stored; entry point URL saved; plugin appears in marketplace listing.
Plugin Activation	1. Activate a plugin for a project. 2. Verify it executes during test runs.	Plugin activated; execution triggered at correct lifecycle hook; PluginExecution record created; output captured.
Marketplace Browsing	1. Browse the marketplace. 2. Install a plugin. 3. Write a review.	Marketplace lists available plugins; install adds plugin to project; review submitted with rating; average rating calculated.

M34: Integration and Polish

Test Item	Test Procedure	Success Criteria
Cross-Feature Integration	1. Run a complete workflow: discover, generate, execute, auto-heal, notify. 2. Verify all features work together.	End-to-end workflow completes; notifications dispatched; credits deducted correctly at each step; intelligence dashboard updates; no data inconsistencies.
Error Boundary Resilience	1. Trigger an error in each major feature area. 2. Verify error boundaries catch it.	Error boundaries display user-friendly messages; "Skip to Dashboard" option available; no blank white screens; error details are logged; no unhandled promise rejections.

Test Item	Test Procedure	Success Criteria
Dashboard Performance	1. Load dashboard with 10+ projects and 50+ test runs. 2. Measure load time.	Dashboard loads within 5 seconds; all panels render; no memory leaks on navigation; API responses under 2 seconds for standard queries.

Phase 5 Completion Criteria

All 6 milestones (M29 through M34) must pass. The end-to-end workflow (M34) is the ultimate validation: every feature must work together without errors. SSO configuration must work with at least one identity provider. The audit chain must be tamper-evident.

Phase Sign-Off	Value
Phase Name	Phase 5: Enterprise Intelligence
Tester Name	_____
Date Tested	____ / ____ / ____
All Milestones Passed	Yes / No
Blockers Found	_____
Signature	_____

Pre-Launch Hardening Checklist

These items must be completed and verified BEFORE inviting external testers. They address the critical security, reliability, and operational gaps identified during the pre-launch audit. Each item is a hard blocker for commercial launch.

Test Item	Test Procedure	Success Criteria
Remove Hardcoded Auth Secret	1. Verify NEXTAUTH_SECRET is set on Vercel. 2. Remove fallback string from src/lib/auth.ts.	App refuses to start without NEXTAUTH_SECRET; no known fallback secret in codebase.
Fix prisma db push --accept-data-loss	1. Create initial migration: npx prisma migrate dev --name init. 2. Change postbuild script to prisma migrate deploy. 3. Deploy and verify.	postbuild uses migrate deploy; no --accept-data-loss flag; production data safe on redeployment.
Fix TypeScript Errors	1. Run npx tsc --noEmit. 2. Fix all errors. 3. Set ignoreBuildErrors: false.	Zero TypeScript errors; build succeeds with strict checking; no silent type violations.
Add Server-Side Auth in Middleware	1. Update src/middleware.ts to check session cookie. 2. Redirect unauthenticated users. 3. Return 401 for unauthorized API requests.	Protected routes require authentication server-side; API routes return 401 without session; no client-only auth enforcement.

Test Item	Test Procedure	Success Criteria
Move Screenshots to Object Storage	1. Set up Vercel Blob or Cloudflare R2. 2. Modify screenshot capture to upload to storage. 3. Store URL in DB instead of base64.	Screenshots stored in object storage; DB stores URLs only; database size does not grow with test screenshots; images accessible via signed URLs.
Configure Stripe Live Mode	1. Complete Stripe account verification. 2. Switch to live keys. 3. Test a real transaction and refund.	Live Stripe checkout works; webhook processes real events; refunds process correctly; Paystack configured for African markets.
Set Up Redis Rate Limiting	1. Create Upstash Redis instance. 2. Replace in-memory rate limiter with Redis-backed. 3. Verify rate limits persist across cold starts.	Rate limits work across Vercel cold starts; per-plan limits enforced; X-RateLimit headers present; 429 responses correct.
Add Error Monitoring (Sentry)	1. Create Sentry project. 2. Install and configure Sentry SDK. 3. Trigger a test error.	Errors captured and reported; source maps uploaded; performance monitoring active; alerts configured for critical errors.
Add Terms of Service and Privacy Policy	1. Create /legal/terms and /legal/privacy pages. 2. Link from footer.	Legal pages accessible; footer links work; GDPR consent banner for EU users; cookie policy included.
Create .env.example	1. Document all required environment variables. 2. Add to repository root.	All 25+ env vars documented with descriptions; required vs optional clearly marked; example values provided (no real secrets); file committed to git.

Phase Sign-Off	Value
Phase Name	Pre-Launch Hardening
Tester Name	_____
Date Tested	____ / ____ / ____
All Milestones Passed	Yes / No
Blockers Found	_____
Signature	_____

Final Launch Sign-Off

This section is the final gate before Probato AI goes live to external testers and then the public. All previous phases and the hardening checklist must be complete. The sign-off below confirms that the platform is ready for commercial operation.

Launch Readiness Check	Status
All Phase 1-5 milestones passed	PASS / FAIL
Pre-launch hardening checklist complete	PASS / FAIL
Stripe live mode processing real payments	PASS / FAIL
Error monitoring active and alerting	PASS / FAIL
No hardcoded secrets in codebase	PASS / FAIL
Database migrations safe (no --accept-data-loss)	PASS / FAIL
Legal pages (Terms, Privacy) published	PASS / FAIL
Custom domain configured and verified	PASS / FAIL
At least 3 external testers invited	PASS / PENDING

Final Launch Authorization	Value
Product Owner	_____
Technical Lead	_____
Launch Date	____ / ____ / ____
Go / No-Go Decision	GO / NO-GO
Outstanding Blockers	_____